TUGAS PRAKTIKUM 4

TUGAS 4 – SISTEM OPERASI

Dosen pengampu

Triawan Adi Cahyanto M.Kom

Disusun Oleh :

Divo Abdil Arkananta(2410651059)

Tugas 1: Eksperimen dengan Data Berbeda

Jalankan setiap algoritma dengan data berikut dan catat hasilnya:

Data Set 1:

P1: AT=0, BT=10

P2: AT=1, BT=5

P3: AT=2, BT=8

Data Set 2:

P1: AT=0, BT=5

P2: AT=1, BT=3

P3: AT=2, BT=8

P4: AT=3, BT=6

```c
#include <stdbool.h>

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
};

void fcfs(struct Process p[], int n) {
    int time = 0;
    float totalWT = 0, totalTAT = 0;

    printf("\n=== FCFS ===\n");
    for (int i = 0; i < n; i++) {
        if (time < p[i].arrival_time)
            time = p[i].arrival_time;
        p[i].waiting_time = time - p[i].arrival_time;
        time += p[i].burst_time;
        p[i].completion_time = time;
        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;

        totalWT += p[i].waiting_time;
        totalTAT += p[i].turnaround_time;

        printf("P%d\tAT=%d\tBT=%d\tCT=%d\tTAT=%d\tWT=%d\n",
               p[i].pid, p[i].arrival_time, p[i].burst_time,
               p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
    }
    printf("Rata-rata WT=%.2f, TAT=%.2f\n", totalWT / n, totalTAT / n);
}

void sjf(struct Process p[], int n) {
    bool done[10] = {false};
    int time = 0, doneCount = 0;
    float totalWT = 0, totalTAT = 0;

    printf("\n=== SJF Non-Preemptive ===\n");
    while (doneCount < n) {
        int idx = -1;
        int minBT = 9999;
        for (int i = 0; i < n; i++) {
            if (!done[i] && p[i].arrival_time <= time && p[i].burst_time < minBT) {
                idx = i;
                minBT = p[i].burst_time;
            }
```

```
        }
        if (idx == -1) {
            time++;
            continue;
        }
        p[idx].waiting_time = time - p[idx].arrival_time;
        time += p[idx].burst_time;
        p[idx].completion_time = time;
        p[idx].turnaround_time = p[idx].completion_time - p[idx].arrival_time;
        done[idx] = true;
        doneCount++;

        totalWT += p[idx].waiting_time;
        totalTAT += p[idx].turnaround_time;

        printf("P%d\tAT=%d\tBT=%d\tCT=%d\tTAT=%d\tWT=%d\n",
               p[idx].pid, p[idx].arrival_time, p[idx].burst_time,
               p[idx].completion_time, p[idx].turnaround_time, p[idx].waiting_time);
    }
    printf("Rata-rata WT=%.2f, TAT=%.2f\n", totalWT / n, totalTAT / n);


oid round_robin(struct Process p[], int n, int quantum) {
    int rem[10], time = 0, completed = 0;
    float totalWT = 0, totalTAT = 0;
    for (int i = 0; i < n; i++) rem[i] = p[i].burst_time;

    printf("\n=== Round Robin (Q=%d) ===\n", quantum);
    while (completed < n) {
        bool done = true;
        for (int i = 0; i < n; i++) {
            if (rem[i] > 0 && p[i].arrival_time <= time) {
                done = false;
                if (rem[i] > quantum) {
                    time += quantum;
                    rem[i] -= quantum;
                } else {
                    time += rem[i];
                    p[i].completion_time = time;
                    p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
                    p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;
                    rem[i] = 0;
                    completed++;
                }
            }
        }
        if (done) time++;
    }
```

```c
    for (int i = 0; i < n; i++) {
        totalWT += p[i].waiting_time;
        totalTAT += p[i].turnaround_time;
        printf("P%d\tAT=%d\tBT=%d\tCT=%d\tTAT=%d\tWT=%d\n",
               p[i].pid, p[i].arrival_time, p[i].burst_time,
               p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
    }

    printf("Rata-rata WT=%.2f, TAT=%.2f\n", totalWT / n, totalTAT / n);
}

int main() {
    struct Process set1[] = {
        {1, 0, 10}, {2, 1, 5}, {3, 2, 8}
    };
    struct Process set2[] = {
        {1, 0, 5}, {2, 1, 3}, {3, 2, 8}, {4, 3, 6}
    };

    printf("\n=== Data Set 1 ===\n");
    fcfs(set1, 3);
    sjf(set1, 3);
    round_robin(set1, 3, 4);

    printf("\n\n=== Data Set 2 ===\n");
    fcfs(set2, 4);
    sjf(set2, 4);
    round_robin(set2, 4, 4);

    return 0;
}
```

Output :

```
divo@MSI:~$ nano scheduling.c
divo@MSI:~$ gcc scheduling.c -o scheduling
divo@MSI:~$ ./scheduling

=== Data Set 1 ===

=== FCFS ===
P1      AT=0    BT=10   CT=10   TAT=10  WT=0
P2      AT=1    BT=5    CT=15   TAT=14  WT=9
P3      AT=2    BT=8    CT=23   TAT=21  WT=13
Rata-rata WT=7.33, TAT=15.00

=== SJF Non-Preemptive ===
P1      AT=0    BT=10   CT=10   TAT=10  WT=0
P2      AT=1    BT=5    CT=15   TAT=14  WT=9
P3      AT=2    BT=8    CT=23   TAT=21  WT=13
Rata-rata WT=7.33, TAT=15.00

=== Round Robin (Q=4) ===
P1      AT=0    BT=10   CT=23   TAT=23  WT=13
P2      AT=1    BT=5    CT=17   TAT=16  WT=11
P3      AT=2    BT=8    CT=21   TAT=19  WT=11
Rata-rata WT=11.67, TAT=19.33


=== Data Set 2 ===

=== FCFS ===
P1      AT=0    BT=5    CT=5    TAT=5   WT=0
P2      AT=1    BT=3    CT=8    TAT=7   WT=4
P3      AT=2    BT=8    CT=16   TAT=14  WT=6
P4      AT=3    BT=6    CT=22   TAT=19  WT=13
Rata-rata WT=5.75, TAT=11.25

=== SJF Non-Preemptive ===
P1      AT=0    BT=5    CT=5    TAT=5   WT=0
P2      AT=1    BT=3    CT=8    TAT=7   WT=4
P4      AT=3    BT=6    CT=14   TAT=11  WT=5
P3      AT=2    BT=8    CT=22   TAT=20  WT=12
Rata-rata WT=5.25, TAT=10.75

=== Round Robin (Q=4) ===
P1      AT=0    BT=5    CT=16   TAT=16  WT=11
P2      AT=1    BT=3    CT=7    TAT=6   WT=3
P3      AT=2    BT=8    CT=20   TAT=18  WT=10
P4      AT=3    BT=6    CT=22   TAT=19  WT=13
Rata-rata WT=9.25, TAT=14.75
divo@MSI:~$ nano scheduling.c
```

Tugas 2: Analisis Eksperimen Tiga Algoritma

1. **Algoritma mana yang memberikan Average Waiting Time terkecil?**
   Algoritma **SJF (Shortest Job First)** menghasilkan rata-rata waktu tunggu paling kecil dibanding yang lain.

2. **Mengapa hasil algoritma berbeda-beda?**
   Karena setiap algoritma punya cara berbeda dalam menentukan urutan proses. FCFS berdasarkan urutan datang, SJF berdasarkan lama proses, sedangkan Round Robin membagi waktu eksekusi secara bergiliran.

3. **Kapan sebaiknya menggunakan algoritma FCFS?**
   FCFS cocok digunakan saat semua proses memiliki durasi hampir sama dan tidak terlalu sensitif terhadap waktu tunggu.

4. **Kapan sebaiknya menggunakan algoritma SJF?**
   SJF baik digunakan ketika waktu eksekusi setiap proses sudah diketahui dan sistem ingin meminimalkan waktu tunggu rata-rata.

5. **Kapan sebaiknya menggunakan algoritma Round Robin?**
   Round Robin cocok untuk sistem **multitasking atau time-sharing**, di mana semua proses harus mendapat jatah waktu CPU yang adil.

Tugas 3: Modifikasi Program

Tambahkan fitur berikut pada salah satu program:

1. Tampilkan Gantt Chart yang lebih detail

2. Simpan hasil ke file text

3. Baca input dari file

```c
#include <stdio.h>
#include <stdbool.h>

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
};

// === Fungsi FCFS dengan Gantt Chart dan Simpan ke File ===
void fcfs(struct Process p[], int n) {
    FILE *f = fopen("hasil.txt", "w");
    int time = 0;
    float totalWT = 0, totalTAT = 0;

    fprintf(f, "=== FCFS Scheduling ===\n");
    printf("=== FCFS Scheduling ===\n");
    printf("\nGantt Chart: |");

    for (int i = 0; i < n; i++) {
        if (time < p[i].arrival_time)
            time = p[i].arrival_time;
        p[i].waiting_time = time - p[i].arrival_time;
        time += p[i].burst_time;
        p[i].completion_time = time;
        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;

        totalWT += p[i].waiting_time;
        totalTAT += p[i].turnaround_time;

        // Tampilkan Gantt Chart di terminal
        printf(" P%d |", p[i].pid);
        fprintf(f, "P%d -> CT=%d, TAT=%d, WT=%d\n",
                p[i].pid, p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
    }

    printf("\n\nRata-rata WT = %.2f\n", totalWT / n);
    printf("Rata-rata TAT = %.2f\n", totalTAT / n);

    fprintf(f, "\nRata-rata WT = %.2f\nRata-rata TAT = %.2f\n",
            totalWT / n, totalTAT / n);
    fclose(f);

    printf("\nHasil juga disimpan ke file 'hasil.txt'\n");
}
```

```c
int main() {
    int n;
    struct Process p[10];

    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Masukkan Arrival Time P%d: ", i + 1);
        scanf("%d", &p[i].arrival_time);
        printf("Masukkan Burst Time P%d: ", i + 1);
        scanf("%d", &p[i].burst_time);
    }

    fcfs(p, n);
    return 0;
}
```

Outputnya :

```
divo@MSI:~$ nano tugas3_scheduling.c
divo@MSI:~$ gcc tugas3_scheduling.c -o tugas3
divo@MSI:~$ ./tugas3
Masukkan jumlah proses: 12
Masukkan Arrival Time P1: 1
Masukkan Burst Time P1: 12
Masukkan Arrival Time P2: 3
Masukkan Burst Time P2: 4
Masukkan Arrival Time P3: 5
Masukkan Burst Time P3: 6
Masukkan Arrival Time P4: 7
Masukkan Burst Time P4: 34
Masukkan Arrival Time P5: 3
Masukkan Burst Time P5: 2
Masukkan Arrival Time P6: 1
Masukkan Burst Time P6: 1
Masukkan Arrival Time P7: 3
Masukkan Burst Time P7: 3
Masukkan Arrival Time P8: 1
Masukkan Burst Time P8: 2
Masukkan Arrival Time P9:
2
Masukkan Burst Time P9: 2
Masukkan Arrival Time P10: 211
Masukkan Burst Time P10: 2
Masukkan Arrival Time P11:
1
Masukkan Burst Time P11: 2
Masukkan Arrival Time P12:
2
Masukkan Burst Time P12: 12
=== FCFS Scheduling ===

Gantt Chart: | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 | P11 | P12 |

Rata-rata WT = 63.08
Rata-rata TAT = 69.92

Hasil juga disimpan ke file 'hasil.txt'
*** stack smashing detected ***: terminated
Aborted (core dumped)
```

```
divo@MSI:~$ nano tugas3_scheduling.c
divo@MSI:~$ gcc tugas3_scheduling.c -o tugas3
divo@MSI:~$ ./tugas3
Masukkan jumlah proses: 12
Masukkan Arrival Time P1: 1
Masukkan Burst Time P1: 12
Masukkan Arrival Time P2: 3
Masukkan Burst Time P2: 4
Masukkan Arrival Time P3: 5
Masukkan Burst Time P3: 6
Masukkan Arrival Time P4: 7
Masukkan Burst Time P4: 34
Masukkan Arrival Time P5: 3
Masukkan Burst Time P5: 2
Masukkan Arrival Time P6: 1
Masukkan Burst Time P6: 1
Masukkan Arrival Time P7: 3
Masukkan Burst Time P7: 3
Masukkan Arrival Time P8: 1
Masukkan Burst Time P8: 2
Masukkan Arrival Time P9:
2
Masukkan Burst Time P9: 2
Masukkan Arrival Time P10: 211
Masukkan Burst Time P10: 2
Masukkan Arrival Time P11:
1
Masukkan Burst Time P11: 2
Masukkan Arrival Time P12:
2
Masukkan Burst Time P12: 12
=== FCFS Scheduling ===

Gantt Chart: | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 | P11 | P12 |

Rata-rata WT = 63.08
Rata-rata TAT = 69.92

Hasil juga disimpan ke file 'hasil.txt'
*** stack smashing detected ***: terminated
Aborted (core dumped)
```

```
divo@MSI:~$ cat hasil.txt
=== FCFS Scheduling ===
P1 -> CT=13, TAT=12, WT=0
P2 -> CT=17, TAT=14, WT=10
P3 -> CT=23, TAT=18, WT=12
P4 -> CT=57, TAT=50, WT=16
P5 -> CT=59, TAT=56, WT=54
P6 -> CT=60, TAT=59, WT=58
P7 -> CT=63, TAT=60, WT=57
P8 -> CT=65, TAT=64, WT=62
P9 -> CT=67, TAT=65, WT=63
P10 -> CT=213, TAT=2, WT=0
P11 -> CT=215, TAT=214, WT=212
P12 -> CT=227, TAT=225, WT=213

Rata-rata WT = 63.08
Rata-rata TAT = 69.92
```

Tugas 4: Eksperimen dengan data berbeda untuk perbandingan algoritma

Test Case 1: Proses dengan burst time bervariasi

Jumlah proses: 4

Time quantum: 3

- P1: AT=0, BT=24

- P2: AT=1, BT=3

- P3: AT=2, BT=3

- P4: AT=3, BT=6

Test Case 2: Proses datang bersamaan

Jumlah proses: 5

Time quantum: 4

- P1: AT=0, BT=10

- P2: AT=0, BT=5

- P3: AT=0, BT=8

- P4: AT=0, BT=12

- P5: AT=0, BT=6

Test Case 3: Proses datang dengan interval

Jumlah proses: 5

Time quantum: 2

- P1: AT=0, BT=8

- P2: AT=2, BT=4

- P3: AT=4, BT=9

- P4: AT=6, BT=5

- P5: AT=8, BT=3

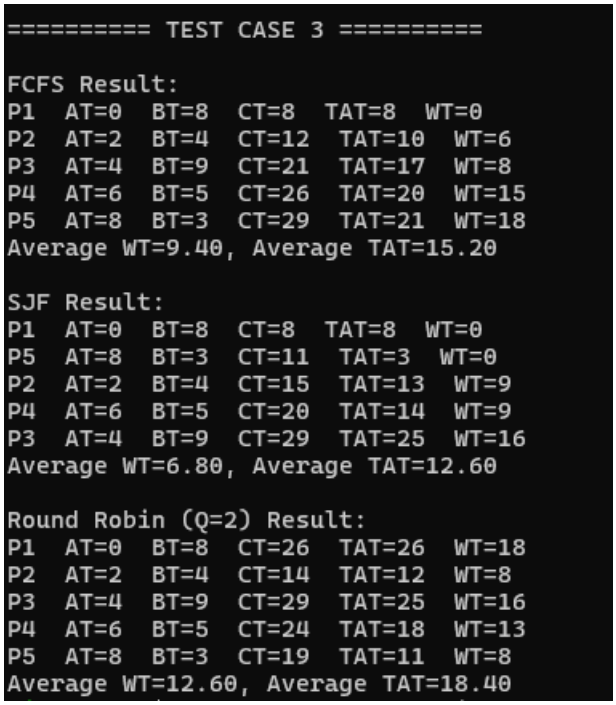
```c
#include <stdio.h>
#include <stdbool.h>

struct Process {
    int pid;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
};

// === FCFS ===
void fcfs(struct Process p[], int n) {
    int time = 0;
    float avgWT = 0, avgTAT = 0;

    for (int i = 0; i < n; i++) {
        if (time < p[i].at)
            time = p[i].at;
        p[i].wt = time - p[i].at;
        time += p[i].bt;
        p[i].ct = time;
        p[i].tat = p[i].ct - p[i].at;

        avgWT += p[i].wt;
        avgTAT += p[i].tat;
    }

    printf("\nFCFS Result:\n");
    for (int i = 0; i < n; i++)
        printf("P%d  AT=%d  BT=%d  CT=%d  TAT=%d  WT=%d\n",
               p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    printf("Average WT=%.2f, Average TAT=%.2f\n", avgWT / n, avgTAT / n);
}

// === SJF Non-Preemptive ===
void sjf(struct Process p[], int n) {
    bool done[10] = {false};
    int time = 0, completed = 0;
    float avgWT = 0, avgTAT = 0;

    printf("\nSJF Result:\n");
    while (completed < n) {
        int idx = -1;
        int minBT = 9999;
        for (int i = 0; i < n; i++) {
            if (!done[i] && p[i].at <= time && p[i].bt < minBT) {
                minBT = p[i].bt;
```

```
        }
        if (idx == -1) {
            time++;
            continue;
        }
        p[idx].wt = time - p[idx].at;
        time += p[idx].bt;
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        done[idx] = true;
        completed++;

        avgWT += p[idx].wt;
        avgTAT += p[idx].tat;

        printf("P%d  AT=%d  BT=%d  CT=%d  TAT=%d  WT=%d\n",
               p[idx].pid, p[idx].at, p[idx].bt, p[idx].ct, p[idx].tat, p[idx].wt);
    }
    printf("Average WT=%.2f, Average TAT=%.2f\n", avgWT / n, avgTAT / n);


/ === Round Robin ===
oid round_robin(struct Process p[], int n, int quantum) {
    int remBT[10];
    for (int i = 0; i < n; i++) remBT[i] = p[i].bt;

    int time = 0, completed = 0;
    float avgWT = 0, avgTAT = 0;

    printf("\nRound Robin (Q=%d) Result:\n", quantum);
    while (completed < n) {
        bool idle = true;
        for (int i = 0; i < n; i++) {
            if (remBT[i] > 0 && p[i].at <= time) {
                idle = false;
                if (remBT[i] > quantum) {
                    time += quantum;
                    remBT[i] -= quantum;
                } else {
                    time += remBT[i];
                    remBT[i] = 0;
                    p[i].ct = time;
                    p[i].tat = p[i].ct - p[i].at;
                    p[i].wt = p[i].tat - p[i].bt;
                    avgWT += p[i].wt;
                    avgTAT += p[i].tat;
                    completed++;
                }
            }
```

```
    for (int i = 0; i < n; i++)
        printf("P%d  AT=%d  BT=%d  CT=%d  TAT=%d  WT=%d\n",
               p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    printf("Average WT=%.2f, Average TAT=%.2f\n", avgWT / n, avgTAT / n);
}

// === Jalankan 3 Test Case ===
void run_all_cases() {
    // Test Case 1
    struct Process case1[] = {
        {1, 0, 24}, {2, 1, 3}, {3, 2, 3}, {4, 3, 6}
    };
    int n1 = 4, q1 = 3;

    printf("\n========== TEST CASE 1 ==========\n");
    fcfs(case1, n1);
    sjf(case1, n1);
    round_robin(case1, n1, q1);

    // Test Case 2
    struct Process case2[] = {
        {1, 0, 10}, {2, 0, 5}, {3, 0, 8}, {4, 0, 12}, {5, 0, 6}
    };
    int n2 = 5, q2 = 4;

    printf("\n========== TEST CASE 2 ==========\n");
    fcfs(case2, n2);
    sjf(case2, n2);
    round_robin(case2, n2, q2);

    // Test Case 3
    struct Process case3[] = {
        {1, 0, 8}, {2, 2, 4}, {3, 4, 9}, {4, 6, 5}, {5, 8, 3}
    };
    int n3 = 5, q3 = 2;

    printf("\n========== TEST CASE 3 ==========\n");
    fcfs(case3, n3);
    sjf(case3, n3);
    round_robin(case3, n3, q3);
}

int main() {
    run_all_cases();
    return 0;
}
```

Ouput :

```
FCFS Result:
P1  AT=0  BT=24  CT=24  TAT=24  WT=0
P2  AT=1  BT=3  CT=27  TAT=26  WT=23
P3  AT=2  BT=3  CT=30  TAT=28  WT=25
P4  AT=3  BT=6  CT=36  TAT=33  WT=27
Average WT=18.75, Average TAT=27.75

SJF Result:
P1  AT=0  BT=24  CT=24  TAT=24  WT=0
P2  AT=1  BT=3  CT=27  TAT=26  WT=23
P3  AT=2  BT=3  CT=30  TAT=28  WT=25
P4  AT=3  BT=6  CT=36  TAT=33  WT=27
Average WT=18.75, Average TAT=27.75

Round Robin (Q=3) Result:
P1  AT=0  BT=24  CT=36  TAT=36  WT=12
P2  AT=1  BT=3  CT=6  TAT=5  WT=2
P3  AT=2  BT=3  CT=9  TAT=7  WT=4
P4  AT=3  BT=6  CT=18  TAT=15  WT=9
Average WT=6.75, Average TAT=15.75

========== TEST CASE 2 ==========

FCFS Result:
P1  AT=0  BT=10  CT=10  TAT=10  WT=0
P2  AT=0  BT=5  CT=15  TAT=15  WT=10
P3  AT=0  BT=8  CT=23  TAT=23  WT=15
P4  AT=0  BT=12  CT=35  TAT=35  WT=23
P5  AT=0  BT=6  CT=41  TAT=41  WT=35
Average WT=16.60, Average TAT=24.80

SJF Result:
P2  AT=0  BT=5  CT=5  TAT=5  WT=0
P5  AT=0  BT=6  CT=11  TAT=11  WT=5
P3  AT=0  BT=8  CT=19  TAT=19  WT=11
P1  AT=0  BT=10  CT=29  TAT=29  WT=19
P4  AT=0  BT=12  CT=41  TAT=41  WT=29
Average WT=12.80, Average TAT=21.00

Round Robin (Q=4) Result:
P1  AT=0  BT=10  CT=37  TAT=37  WT=27
P2  AT=0  BT=5  CT=25  TAT=25  WT=20
P3  AT=0  BT=8  CT=29  TAT=29  WT=21
P4  AT=0  BT=12  CT=41  TAT=41  WT=29
P5  AT=0  BT=6  CT=35  TAT=35  WT=29
Average WT=25.20, Average TAT=33.40
```



Tugas 5: Analisis Eksperimen perbandingan algoritma

Setelah menjalankan program dengan berbagai test case, jawab pertanyaan berikut:

1. Analisis Performa

- **Kapan FCFS terbaik:**
  FCFS memberikan hasil terbaik saat semua proses memiliki waktu eksekusi yang mirip dan datang hampir bersamaan, sehingga tidak ada proses yang menunggu terlalu lama.
- **Mengapa SJF hampir selalu punya Average WT terkecil:**
  Karena SJF selalu memprioritaskan proses dengan burst time paling pendek, otomatis waktu tunggu rata-rata akan lebih kecil dibanding algoritma lain.
- **Trade-off Round Robin:**
  Round Robin adil untuk semua proses, tetapi terlalu sering melakukan pergantian proses (context switch), sehingga bisa menurunkan efisiensi CPU.

Context Switching

- **Algoritma paling banyak context switch:**
  Round Robin memiliki context switch paling banyak karena setiap proses mendapat jatah waktu kecil (quantum).
- **Pengaruh quantum terhadap context switch:**
  Semakin kecil quantum, semakin sering context switch terjadi. Jika quantum diperbesar, jumlah context switch berkurang tapi respon proses jadi lebih lambat.
- **Hasil eksperimen (quantum 2, 4, 8, 16):**
  Dengan quantum kecil (2), CPU sering berpindah antar proses. Saat quantum besar (16), sistem jadi mirip FCFS karena tiap proses bisa selesai tanpa gangguan.

3. Fairness

- **Algoritma paling adil:**
  Round Robin paling adil karena setiap proses mendapat giliran secara bergantian.
- **Proses yang paling dirugikan di FCFS:**
  Proses yang datang belakangan tetapi punya burst time kecil bisa tertunda lama karena harus menunggu proses sebelumnya selesai.
- **Kemungkinan starvation di SJF:**
  Ada. Jika selalu ada proses dengan burst time kecil, proses dengan burst time besar bisa terus tertunda.

4. Real World Application

- **Web server (banyak request kecil):**
  SJF cocok, karena cepat menuntaskan proses ringan lebih dulu.
- **Render video (task besar):**
  FCFS cocok, karena proses besar bisa dijalankan penuh tanpa gangguan.
- **OS desktop (interactive):**
  Round Robin paling cocok, karena memastikan setiap program aktif tetap mendapat respons waktu nyata.